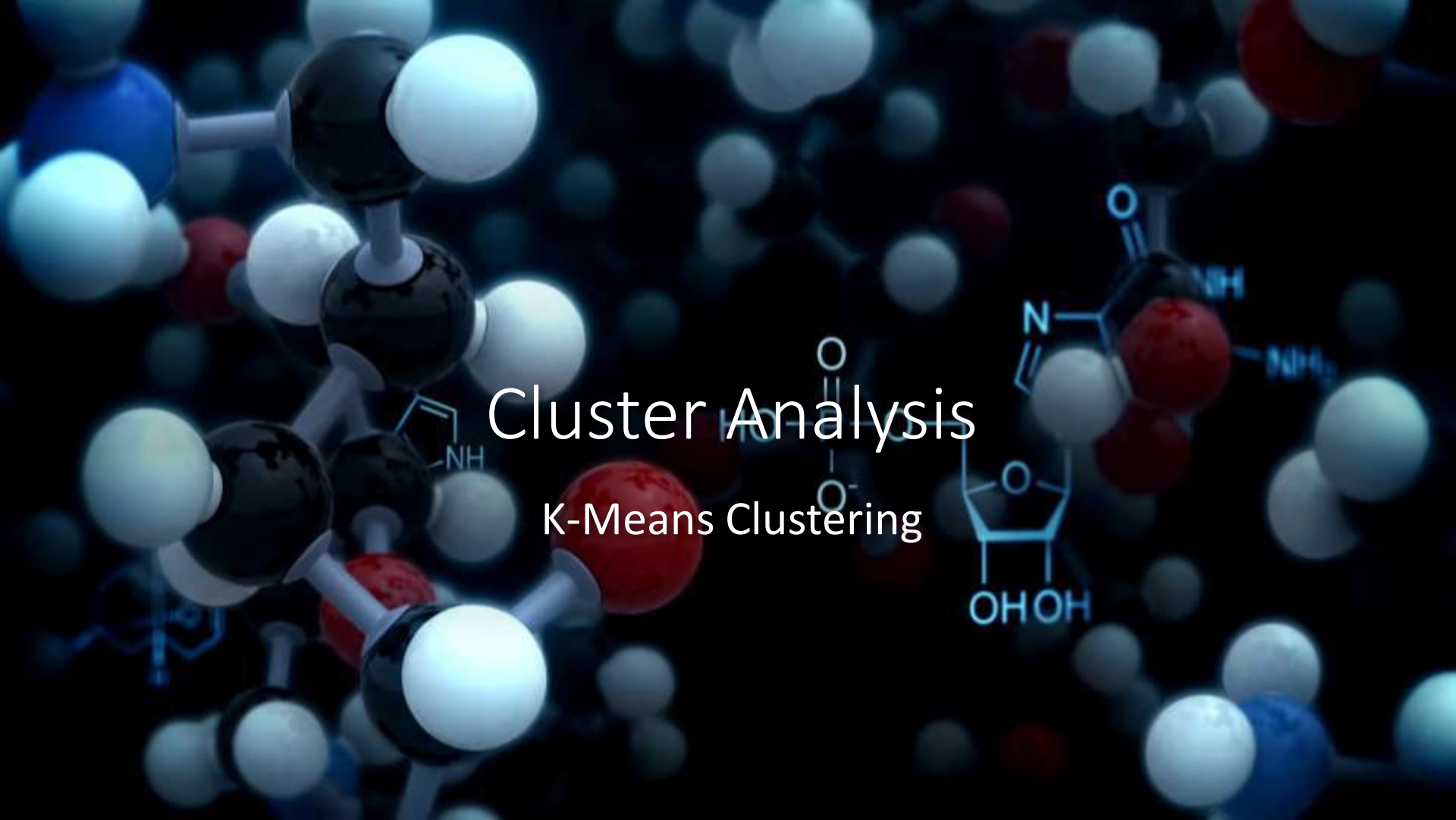


Cluster Analysis

K-Means Clustering

Why Clustering

Clustering can be considered as an intermediate step while analyzing larger volume of data

Clustering has quite a few important applications in data analyses such as

- Detecting homogeneous segments
- Outlier detection
- Group characteristics identification

In many situations, predictive accuracy of a model improves when it is used within clustered data rather than applying on the entire dataset

However, when group information is not available, detecting groups is not an easy task

That is why, several clustering algorithms are proposed

Concept of Clustering

If there are N data points available and they are to be divided into K number of groups, the best way to do this is to put similar data points within a group and dissimilar points into other groups

There are many algorithms available to achieve this goal but partitioning algorithms are the most preferred ones

In partitioning algorithms, the entire dataset is partitioned among K groups to make sure that objects within a cluster are ‘similar’ to each other based on some objective function

In this section, we will focus on the very basic partitioning algorithm, i.e. K-Means clustering

K-Means Clustering

The **k-means algorithm** is an algorithm to cluster *n* objects based on attributes into *k* partitions, where $k < n$

It assumes that the object attributes form a vector space

It is an algorithm for partitioning (or clustering) N data points into K disjoint subsets S_j so as to minimize the within cluster sum-of-squares value. Mathematically, within sum of square is calculated as:

where x_n is a vector representing the the n^{th} data point and u_j is the geometric centroid of the data points in S_j

$$J = \sum_{J=1}^{K} \sum_{n \in S_J} \left| x_n - \mu_J \right|^2$$

Simply speaking k-means clustering is an algorithm to group the objects based on attributes/features into K number of group

K-Means Clustering Algorithm

Step 1: Begin with a decision on the value of k = number of clusters

Step 2: Put any initial partition that classifies the data into k clusters. You may assign the training samples randomly,or systematically as the following

- Take the first k training sample as single- element clusters
- Assign each of the remaining (N-k) training sample to the cluster with the nearest centroid. After each assignment, recompute the centroid of the gaining cluster

K-Means Clustering Algorithm

Step 3: Take each sample in sequence and compute its distance from the centroid of each of the clusters. If a sample is not currently in the cluster with the closest centroid, switch this sample to that cluster and update the centroid of the cluster gaining the new sample and the cluster losing the sample

Step 4 . Repeat step 3 until convergence is achieved, that is until a pass through the training sample causes no new assignments

K-Means Clustering Example (K=2)

Let us consider the tiny dataset given below

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

K-Means Clustering Example (K=2)

- Step 1: Randomly assign any two individuals as two centroids

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Starting Centroids (1.0,1.0) and (5.0,7.0)

K-Means Clustering Example (K=2)

{1,2,3} forms Cluster 1 and {4,5,6,7} forms Cluster 2

Lower Values form cluster

Individual	Centroid 1 Dist	Centroid 2 Dist
1	0.0	7.21
2 (1.5,2.0)	1.12	6.10
3	3.61	3.61
4	7.21	0.0
5	4.72	2.5
6	5.31	2.06
7	4.30	2.93

$$d(m_1,2) = \sqrt{(1-1.5)^2 + (1-2.0)^2} = 1.12$$

$$d(m_2,2) = \sqrt{(5-1.5)^2 + (7-2.0)^2} = 6.10$$

K-Means Clustering Example (K=2)

Individual 3 will move from Cluster 1 to Cluster 2

Thus {1,2} will form Cluster 1 and {3,4,5,6,7} will form Cluster 2

- New Centroids are

$$m_1 = \frac{1}{3}(1+1.5+3), \frac{1}{3}(1+2+4) = (1.83, 2.33)$$

$$m_2 = \frac{1}{4}(5+3.5+4.5+3.5), \frac{1}{4}(7+5+5+4) = (4.12, 5.38)$$

Individual	Centroid 1 Dist	Centroid 2 Dist
1	1.57	7.21
2 (1.5,2.0)	0.47	6.10
3	2.04	1.78
4	5.64	1.84
5	3.15	0.73
6	3.78	0.54
7	2.74	1.08

K-Means Clustering Example (K=2)

Since there is no change in cluster membership, no change in cluster centroid and hence the algorithm stops

Final clusters are {1,2} and {3,4,5,6,7}

- New Centroids are

$$m_1 = \frac{1}{2}(1+1.5), \frac{1}{2}(1+2) = (1.25, 1.5)$$

$$m_2 = \frac{1}{5}(3+5+3.5+4.5+3.5), \frac{1}{5}(4+7+5+5+4) = (3.9, 5.1)$$

Individual	Centroid 1 Dist	Centroid 2 Dist
1	0.56	7.21
2 (1.5,2.0)	0.56	6.10
3	3.05	1.42
4	6.66	2.20
5	4.16	0.42
6	4.78	0.61
7	3.75	0.72

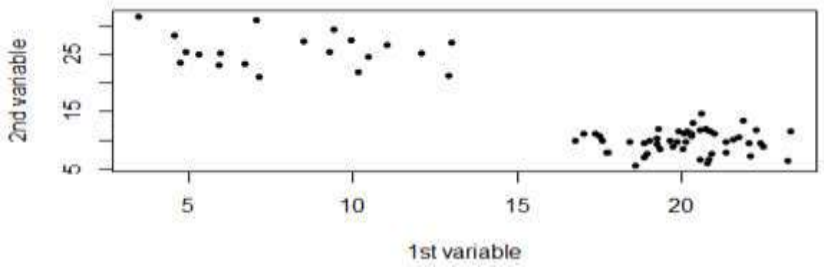
K-Means clustering with R

Let us create some data

```
exampleData=matrix(c(rnorm(50,20,2),rnorm(20,8,3),rnorm(50,
  10,2),rnorm(20,25,3)),nrow=70)
```

The data created in this way has 70 rows and are already segmented in two parts

The scatter plot shows the data distribution



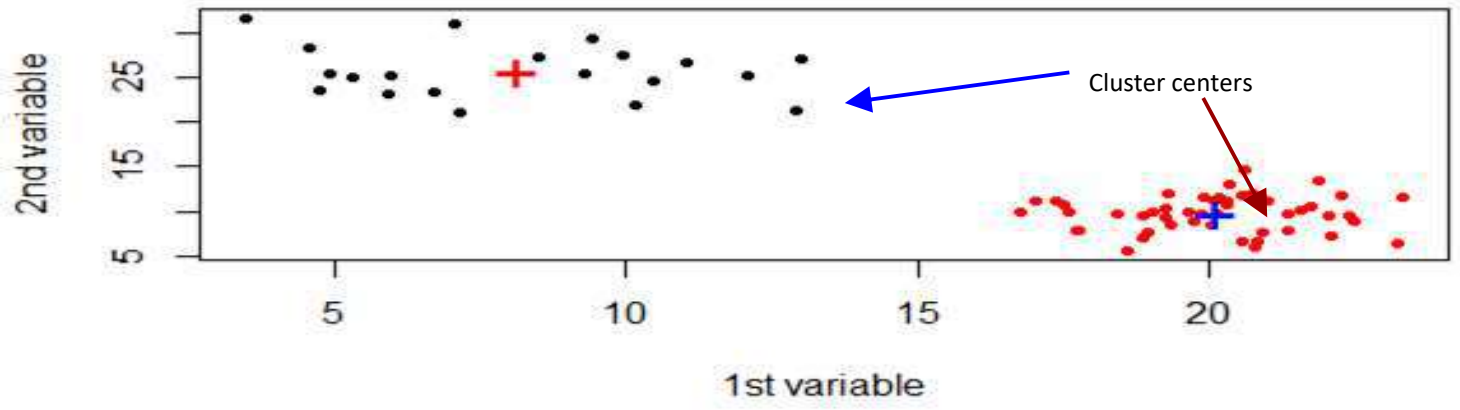
R Code: `plot(exampleData,pch=20,xlab="1st variable",ylab="2nd variable")`

Get the raw data from here: https://www.dropbox.com/s/y3qxg3nv7yqr9oq/exampleData.csv?dl=0
Make sure to remove the 1st column

To run kmeans clustering in R **kmeans()** function is used

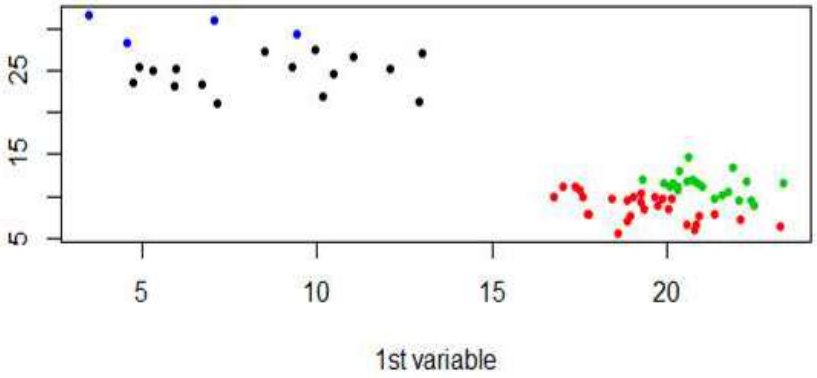
R codes are given below:

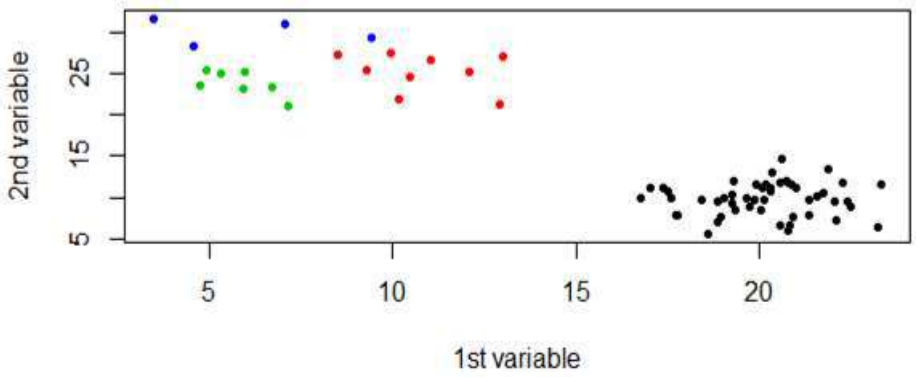
```
cluster=kmeans(exampleData,centers=2)
plot(exampleData,pch=20,xlab="1st variable",ylab="2nd
 variable",col=cluster$cluster)
points(cluster$centers,pch="+",col=c("red","blue"),cex=2
 )
```



Until and unless the clusters are markedly distinct from each other, kmeans clustering will produce different results on each run

If number of clusters are 4 (instead of 2), in two separate runs two separate outcomes have come up (as shown below)

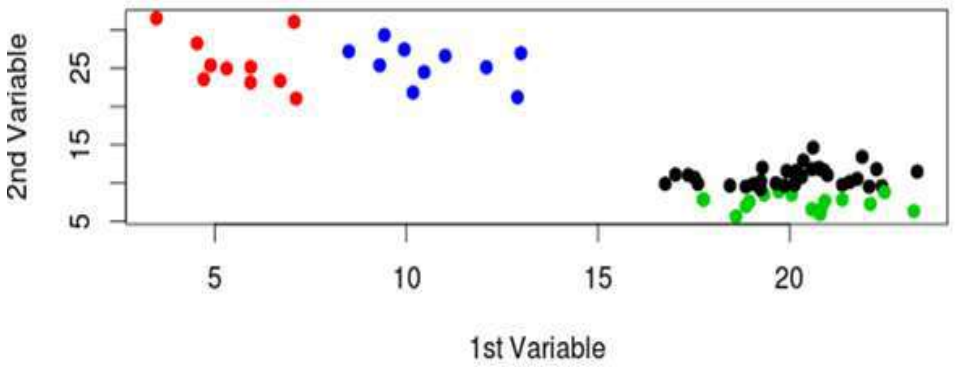




K-means algorithm gives different results in each run because it is a ‘greedy’ algorithm and has very high probability to get stuck into local minima since initialization starts at random

To minimize the chances of getting stuck in local minima, initialization can be started at multiple points (multiple random start) and the best model is returned out of those many random starts

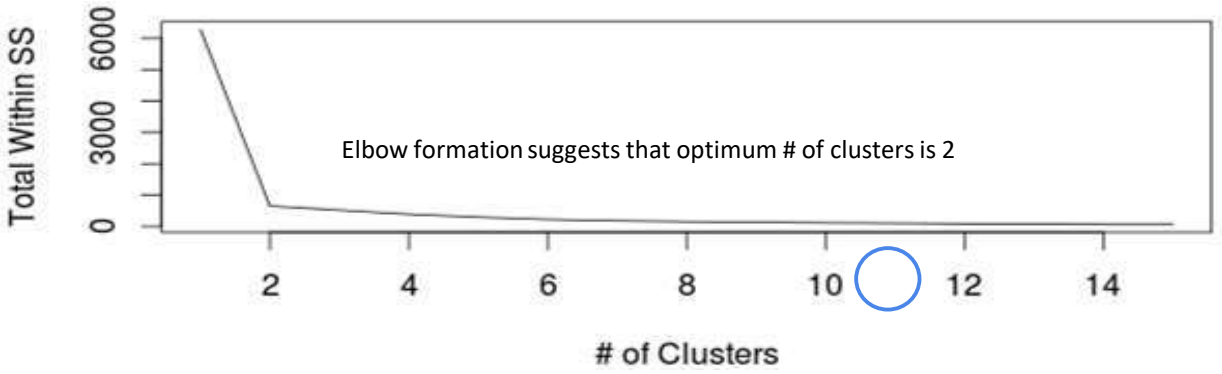
```
fit=kmeans(exampleData,centers=4,nstart=30)
plot(exampleData,pch=19,col=fit$cluster,xlab=”1st Variable”, ylab=”2nd Variable”)
```



Deciding the optimum number of clusters is a concern in K-means clustering as the researcher has to specify the number of clusters to extract apriori

R sends out a few more information as output of k-means clustering and total within sum of square value is one of them which can be used to find out optimum number of clusters

```
wss=sapply(1:15,function(x)kmeans(exampledata,centers=x,nstart=30)$tot.withinss)
plot(1:15,wss,type="l",xlab="# of Clusters",ylab="Total Within SS")
```



In many cases the scree plot does not have a sharp elbow

In those cases ‘silhouette’ distance can be used for finding the optimum number of clusters

Cluster solution, which gives largest average silhouette distance would be considered as the best solution

Usually, if average silhouette distance is above 0.5, a cluster solution is considered as fair and if it crosses 0.7, cluster solution would be considered as good

R Code to extract average silhoutte distance

```
find_silhouette=function(data,cluster){
  require(cluster)
  fit=kmeans(scale(data),centers = cluster,nstart=100)
  s=silhouette(x = fit$cluster,dist(scale(data)))
  mean(s[,3])
}

sapply(2:20,find_silhouette,data=exampleData)
```

Issues and advantages

Advantages

- Very simple algorithm
- Always converges (even though locally)
- Quite fast and interpretation of clusters is quite easy

Issues

- Greatly affected by extreme values
- Performs poorly for irregular shaped data points (longitude and latitude)
- Each time it is run, different results may come out!
- Cannot handle categorical data